

MEMO: Fine-grained Tensor Management For Ultra-long Context LLM Training

PINXUE ZHAO*, Peking University, China
HAILIN ZHANG*, Peking University, China
FANGCHENG FU*, Peking University, China
XIAONAN NIE*, Peking University, China
QIBIN LIU, Tencent Inc., China
FANG YANG, Tencent Inc., China
YUANBO PENG, Tencent Inc., China
DIAN JIAO, Tencent Inc., China
SHUAIPENG LI, Tencent Inc., China
JINBAO XUE, Tencent Inc., China
YANGYU TAO, Tencent Inc., China
BIN CUI*, Peking University, China

Nowadays, Large Language Models (LLMs) have been trained using extended context lengths to foster more creative applications. However, long context training poses great challenges considering the constraint of GPU memory. It not only leads to substantial activation memory consumption during training, but also incurs considerable memory fragmentation. To facilitate long context training, existing frameworks have adopted strategies such as recomputation and various forms of parallelisms. Nevertheless, these techniques rely on redundant computation or extensive memory communication, resulting in low Model FLOPS Utilization (MFU). In this paper, we propose MEMO, a novel LLM training framework designed for fine-grained activation memory management. Given the quadratic scaling of computation and linear scaling of memory with sequence lengths when using FlashAttention, we offload memory-consuming activations to CPU memory after each layer's forward pass and fetch them during the backward pass. To maximize the swapping of activations without hindering computation, and to avoid exhausting limited CPU memory, we implement a token-wise activation recomputation and swapping mechanism. Furthermore, we tackle the memory fragmentation issue by employing a bi-level Mixed Integer Programming (MIP) approach, optimizing memory reuse across transformer layers. Empirical results demonstrate that MEMO achieves an average of $1.97\times$ and $1.80\times$ MFU compared to Megatron-LM and DeepSpeed, respectively. This improvement is attributed to MEMO's ability to

*Pinxue Zhao, Hailin Zhang, Fangcheng Fu, Xiaonan Nie and Bin Cui are with the School of Computer Science & Key Lab of High Confidence Software Technologies (MOE), Peking University. Bin Cui is also with the Institute of Computational Social Science, Peking University (Qingdao).

Authors' Contact Information: Pinxue Zhao, Peking University, China, pinxue.zhao@pku.edu.cn; Hailin Zhang, Peking University, China, z.hl@pku.edu.cn; Fangcheng Fu, Peking University, China, ccchengff@pku.edu.cn; Xiaonan Nie, Peking University, China, xiaonan.nie@pku.edu.cn; Qibin Liu, Tencent Inc., China, brendenliu@tencent.com; Fang Yang, Tencent Inc., China, youngfyang@tencent.com; Yuanbo Peng, Tencent Inc., China, yuanbopeng@tencent.com; Dian Jiao, Tencent Inc., China, focusjiao@tencent.com; Shuaipeng Li, Tencent Inc., China, shuaipengli@tencent.com; Jinbao Xue, Tencent Inc., China, jinbaoxue@tencent.com; Yangyu Tao, Tencent Inc., China, brucetao@tencent.com; Bin Cui, Peking University, China, bin.cui@pku.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/2-ART53
<https://doi.org/10.1145/3709703>

minimize memory fragmentation, reduce recomputation and intensive communication, and circumvent the delays associated with the memory reorganization process due to fragmentation. By leveraging fine-grained activation memory management, MEMO facilitates efficient training of 7B LLM with 1 million sequence length on just 8 A800 GPUs, achieving an MFU of 52.30%.

CCS Concepts: • **Computing methodologies** → **Distributed computing methodologies**.

Additional Key Words and Phrases: Large Language Model, Long Context Training, Tensor Management, Data Rematerialization

ACM Reference Format:

Pinxue Zhao, Hailin Zhang, Fangcheng Fu, Xiaonan Nie, Qibin Liu, Fang Yang, Yuanbo Peng, Dian Jiao, Shuaipeng Li, Jinbao Xue, Yangyu Tao, and Bin Cui. 2025. MEMO: Fine-grained Tensor Management For Ultra-long Context LLM Training. *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 53 (February 2025), 28 pages. <https://doi.org/10.1145/3709703>

1 Introduction

Since the advent of ChatGPT [58], Large Language Models (LLMs) have demonstrated remarkable proficiency in comprehending and generating natural language texts. Besides revolutionizing the field of language processing, which encompasses translation [103], coding [23, 72, 94], etc., transformer-based LLMs have also found applications in multi-modal scenarios, such as image processing [15, 61], video stream analysis [73], and AI for science [2, 5]. To accommodate novel applications that require lengthy contexts [98], LLMs have developed to support long context input, from 2K-4K [79, 81] to 32K [29, 80], 128K [18, 58], or even millions of tokens [1, 9, 41]. Considering the extrapolation problem [43, 66], which refers to the decline in LLM performance when input sequences exceed the training length, it is necessary to conduct long context training [7, 17, 28] or fine-tuning [14, 62] to facilitate long sequence inference. Beyond natural language processing, increasing the context length is also essential across diverse domains, including video processing [101], protein properties prediction [6], weather forecasting [54], and health care [40].

Maximizing system performance with limited memory is a common and significant challenge in the data management community. Within this context, training LLMs with long sequence lengths poses difficulties due to restricted GPU memory. During training, a large amount of activations¹ must be stored for gradient computation during the backward pass, resulting in substantial memory consumption. Typically, it is well known that the self-attention module in the transformer architecture has a quadratic computation and memory complexity w.r.t. the sequence length. FlashAttention [11, 12], now a standard technique for attention computation in LLM training, accelerates computation and shrinks the memory complexity to be linear w.r.t. the sequence length by scheduling memory I/O and recomputing necessary components during the backward pass. Except for attention, the remaining activation memory also scales linearly with the sequence length, which can become quite large in long context scenarios. For instance, training a GPT model with 7B parameters on a sequence length of 1 million can lead to an activation memory of 4096GB, far exceeding the memory capacity of commonly used accelerators (e.g. 80GB for an NVIDIA H100/A100 GPU).

Moreover, dynamic memory allocators inherently face the issue of memory fragmentation due to frequent allocation and deallocation, which complicates efficient data management. Besides storing the *skeletal activations* for the backward pass, there are also tremendous *transient activations* that are temporarily generated during computation (we will formally categorize the two kinds of activations

¹In neural network training, the outputs of operators are referred to as activations. Some of these, termed “skeletal activations” in this work, must be stored for gradient computation during the backward pass, while others are termed “transient activations”. More details are provided in Section 3.

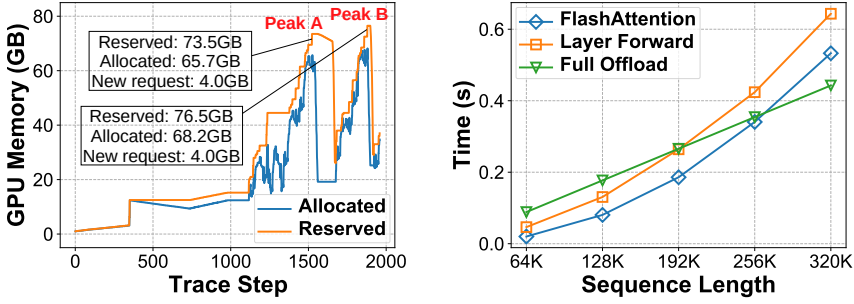


Fig. 1. The left figure, generated using PyTorch’s snapshot API [10], shows the allocated and reserved GPU memory of PyTorch when training a 7B GPT model with sequence length 512K. The right figure shows the time consumption of FlashAttention computation, one transformer layer forward computation, and one-layer full activation offloading when training a 7B GPT on 8 A800 GPUs with a TP size of 8.

in Section 3). Such transient activations have distinct data life cycles and usually lead to frequent allocation and deallocation of GPU memory. Currently, most LLM training systems are built on top of PyTorch [60], including Megatron-LM [77] and DeepSpeed [69]. PyTorch employs a caching memory allocator designed to reduce the costly “cudaMalloc” and “cudaFree” operations by caching and reusing allocated memory blocks. However, the frequent memory (de)allocation requests in the caching allocator result in significant memory fragmentation [22]. This issue becomes more severe in long context training, considering the fact that the (de)allocated memory blocks are significantly larger than those in normal tasks. Memory fragmentation not only leads to Out-Of-Memory (OOM) error but also significantly hinders training efficiency because of the frequent triggering of the PyTorch memory reorganization process, which involves calls to “cudaFree” and “cudaMalloc” to release cached blocks and reclaim GPU memory. Figure 1(a) illustrates an example of GPU memory fragmentation. At the peaks of the curves, there is more than 4GB memory reserved but not allocated. However, when the training task tries to allocate 4GB memory, the allocator fails to find a continuous memory space to fulfill the allocation request. Consequently, it necessitates invoking a series of “cudaFree” and “cudaMalloc” to reorganize memory, which blocks GPU computation.

In this paper, we aim to tackle the memory challenges encountered during long context LLM training. Specifically, we propose and implement an LLM training framework MEMO to address the activation data management problem. There are several key observations that inspire our design.

In the data management domain, when high-bandwidth memory is limited, rematerialization is a typical technique to free up memory by releasing data that is not immediately needed and reconstructing data structures on demand. For instance, Spark [91] supports rematerializing Resilient Distributed Datasets (RDDs) [90] through recomputation or swapping² between CPU memory and disk storage when RAM is insufficient. Faiss [30], a widely used vector database, utilizes a CPU-GPU memory hierarchy to accommodate the high memory requirement. Additionally, the dynamic view materialization solution [64] employs an LRU cache to manage materialized views, and rematerializes cache-missed views upon needed. These methods leverage recomputation and swapping strategies to efficiently manage data structures within the memory hierarchy. Similarly, in deep learning training, to reduce the peak memory consumption caused by skeletal activations,

²Swapping refers to moving data between different levels of memory hierarchy. More details are provided in Section 2.2.

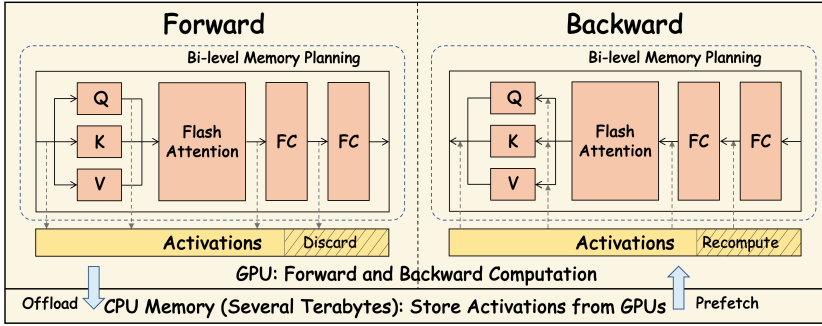


Fig. 2. An overview of MEMO. We devise a fine-grained recomputation and swapping mechanism to manage the skeletal activations for backward propagation, and leverage a bi-level memory planning method to reuse the memory space of transient activations across the transformer layers.

activation recomputation [8, 34, 35] and swapping [56, 71] are also widely adopted.³ Typically, both of them reduce memory consumption at the price of extra time cost. The activation recomputation technique discards some activations in the forward pass and later recomputes them in the backward pass, leading to extra computation cost. The swapping technique offloads the activations to CPU memory in the forward pass to relieve the GPU memory pressure, and later fetches them back to GPU memory in the backward pass, incurring the overhead of data transmission between CPU and GPU memory.

Observation 1: Opportunity for activation swapping. Contemporary mainstream LLM training frameworks such as Megatron-LM and DeepSpeed prefer activation recomputation to swapping, which is due to the fact that the GPU computing ability has a far more rapid growth than the connectivity between CPU and GPU memory in the past few years (see Section 2.2 for details). However, we find that the situation is a bit different in long context training of LLMs. Denote s as the sequence length. The computation complexity of one transformer layer is $O(s^2)$, while the activation memory complexity is $O(s)$ thanks to FlashAttention. During GPU computation, we can leverage the idle CPU-GPU bandwidth, offloading activations to CPU memory during the forward pass, and fetching the activations during the backward pass. As the sequence length increases, there is greater potential for overlapping computation and communication, given that their time requirements scale quadratically and linearly with the sequence length, respectively. As shown in Figure 1(b), eventually, after reaching a specific sequence length (192K in this case), the transmission of activations can be fully overlapped with GPU computation.

However, in practice, there is limited chance to swap all activations. On the one hand, extremely long training data is rare, and most of the time we need to train on data that doesn't fully overlap the activation transmission and the computation. On the other hand, offloading all activations may cause CPU OOM issues — the CPU memory is responsible for storing all activations from all GPUs on the same machine, but the current CPU memory is typically several terabytes, which is insufficient for very long sequence lengths. Considering the above challenges, we introduce a **fine-grained activation recomputation and swapping mechanism** to manage the skeletal activations. We consider both tensor-level and token-level activation management. For each layer, following previous works [37, 56], we consistently offload two activation tensors, the input of each transformer layer and the output of FlashAttention, to CPU memory. For other activation

³Parallelism techniques like sequence parallelism [28, 35] and context parallelism [37, 42, 55] are also compelling approaches to reduce memory at the price of extra communication overhead. Our work is compatible with these parallelism techniques.

tensors, we only offload a fraction (denoted as α) of tokens, and recompute the rest part during the backward pass. We model the time cost of activation recomputation and transmission and determine the fraction α through a well-formulated linear programming problem, which aims to maximize offloading activations without impeding GPU computation or causing CPU OOM issues. During the backward pass, prefetching activations can also overlap with GPU computation, because the backward computation is typically twice as much as the forward computation. With both tensor-level and token-level activation management, we make full use of the idle bandwidth and minimize the recomputation overhead to improve the overall efficiency.

Regarding memory fragmentation, research has utilized the characteristics of targeted workloads to analyze and resolve the issue, such as experimentally analyzing the impact of memory allocation on high-performance query engines [16] and addressing persistent memory fragmentation with efficient defragmentation algorithms [59]. In the same vein, for long-sequence LLM training, we aim to leverage the specific characteristics of LLM training to address memory fragmentation.

Observation 2: Deterministic memory (de)allocation pattern across iterations and layers.

In long sequence LLM training, the memory fragmentation mainly comes from frequent and irregular memory (de)allocation requests. However, we observe that, typical LLM training adheres to a deterministic computation process across iterations and layers. All transformer layers in an LLM are identical, and each training iteration involves the same computation. While the general-purpose caching allocator is designed for dynamic computation routines, training LLMs can be conceptualized as static computation graphs [3], which have identical structures across layers. This provides an opportunity to design static planning for each layer and reuse the allocated memory of each layer, thereby mitigating memory fragmentation.

To enhance memory utilization while minimizing fragmentation, we leverage a **hierarchical Mixed Integer Programming (MIP)** technique to tackle the memory planning problem. Before training, we profile the memory (de)allocation requests of one training iteration, then use MIP to solve an optimized memory plan for a single transformer layer. Since the memory requests of the transformer layers are identical, the entire memory block for one layer can be directly reused for the subsequent identical layer. Considering each transformer layer's memory block as a single memory allocation request, we further solve another MIP problem that plans memory allocation for the entire LLM training, including the initial embedding layer, all transformer layers, and the final classifier layer. We only need to solve the problem once before the actual training, since all iterations can utilize the same memory plan. The near-optimal memory plan eliminates the fragmentation issue and avoids PyTorch's time-consuming memory reorganization mechanism.

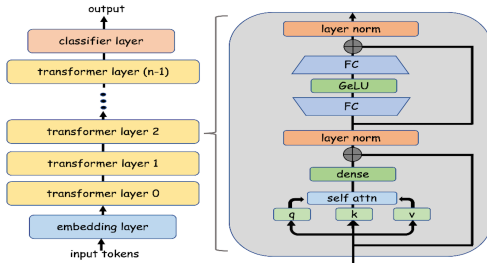
Putting them together, in response to the activation memory challenge in long context training, we propose **MEMO**, an LLM training framework with fine-grained tensor memory management. We consider the challenge as an activation data management problem, and draw inspiration from the data rematerialization and memory defragmentation techniques to address the challenge. Figure 2 presents an overview of MEMO. To make full use of the idle CPU-GPU bandwidth during training with different sequence lengths, we introduce a token-wise fine-grained activation recomputation and swapping strategy. We employ a bi-level hierarchical MIP technique to solve the memory planning problem and eliminate memory fragmentation. To the best of our knowledge, this is the first training framework that enables efficient training of a 7B LLM on 8 GPUs with a sequence length of 1 million.

We summarize our contributions as follows:

- We propose and implement an LLM training framework MEMO to address the activation data management problem in long context LLM training.

Table 1. Commonly used notations in this work.

Notation	Explanation
b	Batch size
s	Context length
n	Number of transformer layers
h	Hidden size
P	Number of model parameters
α	The fraction of swapping



(a) The architecture of a typical LLM.

Forward Memory Request				Backward Memory Request			
index	instruction	tensor_id	size	index	instruction	tensor_id	size
0	malloc	13	128MB	12	malloc	20	512MB
1	malloc	14	128MB	13	free	20	512MB
2	free	14	128MB	14	malloc	21	1024MB
3	malloc	15	256MB	15	malloc	22	256MB
4	free	13	128MB	16	free	15	256MB
5	malloc	16	512MB	17	malloc	23	128MB
6	malloc	17	128MB	18	malloc	24	512MB
7	malloc	18	128MB	19	free	21	1024MB
8	malloc	19	256MB	20	free	22	256MB
9	free	17	128MB	21	free	23	128MB
10	free	19	256MB	22	free	24	512MB
11	free	18	128MB	23	free	16	512MB

(b) An example memory request sequence.

Fig. 3. (a): The architecture of a typical LLM. (b): An example memory request sequence of a transformer layer's forward and backward pass. Tensors 15 and 16 are skeletal tensors, while the others are transient tensors.

- We introduce a fine-grained activation recomputation and swapping mechanism to fully utilize the idle CPU-GPU communication bandwidth during time-consuming GPU computation.
- We employ a bi-level MIP technique to solve the memory planning problem and significantly mitigate memory fragmentation.
- We evaluate MEMO through extensive experiments, and demonstrate an average of 1.97 $\times$ and 1.80 $\times$ improvement in terms of MFU⁴ compared to Megatron-LM and DeepSpeed, respectively. Additionally, MEMO is the first framework that enables the efficient training of 7B LLM with 1 million context length on only 8 A800 GPUs.

2 Preliminary

In this section, we present an overview of the architecture and training process of LLMs, along with memory reduction strategies and distributed training techniques. Commonly used notations are listed in Table 1.

2.1 Large Language Models

2.1.1 Architecture. As shown in Figure 3(a), the architecture of an LLM comprises an input embedding layer, multiple transformer layers, and a final classifier layer. The embedding layer converts input tokens into continuous representations. Each decoder-only transformer layer constitutes a multi-head self-attention module with causal mask, and an Feed-Forward Network (FFN) module

⁴MFU (Model FLOPs Utilization) is a widely used efficiency metric that evaluates how well the accelerators are utilized in model training. It is calculated as the ratio of the observed throughput to the theoretical throughput which assumes the hardware operates at peak Floating-point Operations Per Second (FLOPS) [65]. More details are provided in Section 5.1.

containing Fully-Connected (FC) networks. The classifier layer takes the hidden states produced by the transformer layers as input, and generates a probability distribution over the vocabulary.

2.1.2 The Training Process. The training process of LLM involves two phases: the forward pass and the backward pass. During the forward pass, the model processes the input data through its layers, and finally generates predictions. The output tensors of the operators in forward pass are called activation tensors, some of which are stored for backward pass computation according to gradient-based learning.

The backward pass, on the other hand, computes the gradients with regard to the model parameters. These gradients are used to update the model's parameters. Following the chain rule in gradient computation, the backward pass relies on the activation tensors from the forward pass to compute gradients.

2.1.3 The Challenge of Huge Memory Requirement in Long Context Training. Self-attention is the most critical module in LLMs. It facilitates information interaction between tokens: the input tensor is first projected into query (Q), key (K), and value (V), each with the shape (b, s, h) ; then the output is given by $O = \text{softmax}(QK^T / \sqrt{d}) \cdot V$, which incurs $O(s^2)$ time and space complexity due to the (b, s, s) matrices. FlashAttention [11, 12], the de-facto attention implementation in nowadays LLM computation, processes the computation in tiles, discards intermediate results, and maintains compact states to generate the final output O . This method avoids storing the $O(s^2)$ matrices. During the backward pass, FlashAttention re-computes these intermediate results in a tiled manner for gradient calculation. Thanks to this design, FlashAttention significantly reduces memory requirements to just $O(s)$ complexity. Although several alternative approaches like sparsification [48] (e.g., BigBird [92], Longformer [4]), kernelization (e.g., LinearAttn [32], CosFormer [67]), and low-rank approximation (e.g., Linformer [85], Nyströmformer [87]) also aim to reduce the quadratic memory demands of self-attention, these methods modify the attention mechanism and can potentially compromise accuracy. In this paper, we adopt FlashAttention as the default method, considering its prevalent use in practice.

Although FlashAttention has reduced the memory complexity of LLM training from $O(s^2)$ to $O(s)$, the linearly scaling activation memory remains the primary challenge in long context training. For example, as we will elaborate in Section 3, when training a 7B GPT model with 32 layers and a hidden size of 4096, using a single 1 million length sequence, the forward activation tensors required by the backward pass consume 4096GB (when using half-precision numbers), whereas the typical memory capacity of a GPU is much smaller. To cope with this issue, there are two lines of efforts, which are the memory reduction techniques and distributed parallelism strategies. In the rest of this section, we will introduce these two lines respectively. It is worth noting that although our work primarily concentrates on the memory reduction techniques, the proposed MEMO framework is compatible with a wide range of parallelism strategies.

2.2 Memory Reduction Techniques

As mentioned in Section 1, when existing query processing engines face the challenge of limited high-bandwidth memory, a common strategy to mitigate memory pressure involves discarding certain data structures and rematerializing them as needed [30, 64, 91]. There are two prevalent techniques: (1) recomputing the results and (2) swapping data to lower-tier memory and retrieving it when necessary. Both methods are also widely-used in neural network training to rematerialize activation tensors.

Activation recomputation [8, 34, 35] (a.k.a. activation checkpointing) selectively stores the inputs of certain layers rather than all intermediate activations. During the backward pass, the

required activations are recomputed on-the-fly. While this approach reduces the activation memory footprint required for LLM training, it introduces additional computation, which impacts efficiency. Swapping [56, 70, 71], also known as CPU offloading, aims to relieve the GPU memory pressure by offloading GPU tensors to CPU memory, and fetch them back to GPU when needed. Through careful scheduling, the data transmission overhead can be overlapped with GPU computation, a technique also popular in GPU databases [30, 74, 84]. However, if data transmission is too time-consuming to overlap, swapping can significantly slow down training. In general, both memory reduction techniques release the memory of activations in the forward pass, but need to rematerialize them in the backward pass, at the price of extra computation or data transmission overhead, respectively.

In the past few years, GPU computing capabilities have improved over $100\times$ (e.g., the half-precision performance of P100 and H100 are 18.7 and 1979 TFLOPS, respectively), while the improvement of CPU-GPU bandwidth is only $4\times$ (from PCIe 3.0 to PCIe 5.0). As a consequence, mainstream LLM training frameworks favor the activation recomputation technique.⁵ In practice, when training LLMs with long context input, full activation recomputation is often employed, which involves storing only the input tensor of each transformer layer and recomputing the required activations during backward propagation.

2.3 Distributed Parallelism Strategies

Distributed training is essential for efficiently training LLMs, especially in scenarios of long context training. To facilitate the training of large-scale data and model, several distributed parallelism strategies have been proposed.

Data Parallelism (DP) [13, 38, 104] duplicates model parameters and distributes the input data across multiple devices. Each device holds a complete copy of the model and processes its input data independently. After backward propagation, the devices synchronize parameter gradients to ensure consistency across the model copies.

Zero-Redundancy Optimizer (ZeRO) [68] is a series of variants built upon DP, aiming to alleviate memory pressure. Naive DP replicates model parameters, gradients and optimizer states among all devices. ZeRO is designed in three stages to reduce these memory requirements respectively. First, ZeRO-1 partitions the optimizer states among all DP workers. Next, ZeRO-2 extends ZeRO-1 by also partitioning gradients, further reducing memory footprint. Finally, ZeRO-3, based on ZeRO-2, partitions model parameters among DP workers, further mitigating memory pressure but introducing additional communication to gather parameters during training.

Tensor Parallelism (TP) [77] partitions the self-attention and feed-forward modules of transformer layers across multiple devices along either the column or row dimension. It addresses the problem that LLMs can not fit into the memory of a single device. It involves extra collective communication operations (i.e. AllReduce) to synchronize the intermediate results. Therefore, TP is usually applied within a computing node, where intra-node GPUs are connected via high-bandwidth NVLink.

Pipeline Parallelism (PP) [21, 26, 53] is also proposed to address the problem that LLMs cannot be fit into a single device. Different from TP, PP partitions model layers into several stages, then distributes the stages to different devices. The input data is processed through these stages in a pipeline fashion. Given the peer-to-peer communication style, the PP stages are often distributed across nodes. However, PP introduces a phenomenon known as “bubble”, which corresponds to GPU idle time. The issue becomes more severe when the number of micro-batches is small.

⁵Both Megatron-LM and DeepSpeed have supported activation recomputation for long. Nevertheless, Megatron-LM does not support swapping until the release of TransformerEngine v1.3 in Feb 2024. Besides, DeepSpeed primarily focuses on swapping of model states, encompassing model parameters, gradients and optimizer states [70], as they constitutes the most significant portion of memory footprint in short context training tasks. However, in long context training scenarios, the memory consumption of activations has surpassed that of model states.

To facilitate efficient long context training, several novel parallelism strategies have been proposed recently.

Sequence Parallelism (SP) [35] is built upon TP to further reduce activation memory overhead. It splits the sequence dimension in the part of the model that does not apply TP. The original AllReduce communication now transitions to AllGather and ReduceScatter.

DeepSpeed-Ulysses [28], built upon ZeRO, is another form of sequence parallelism. During self-attention computation, it splits the head dimension, whereas in other model components, it partitions the sequence dimension. For transitioning between modules, it utilizes AllToAll communications, theoretically reducing communication overhead compared to SP. However, its SP degree is limited by the number of heads in self-attention. To further relieve the memory pressure, DeepSpeed-Ulysses leverages ZeRO to distribute model parameters.

Context Parallelism (CP) [37, 42, 55] shards the query, key, and value matrices within the attention module along the sequence dimension across different devices. During attention computation, necessary communications are involved to ensure consistent results, which can be overlapped with computation by careful scheduling.

In practice, these parallelism strategies and memory reduction techniques can be integrated and employed simultaneously to facilitate efficient training of LLMs.

3 Anatomy and System Desiderata

Managing fragmented massive data storage is a critical data management issue, particularly when workloads are constrained by limited high-bandwidth memory. Long-context LLM training demonstrates such a challenge, where huge fragmented activation memory significantly impedes efficient training within constrained GPU memory resources.

In this section, we first provide an in-depth anatomy of the key characteristics of activation data storage in long-context LLM training. Based on this analysis, we present the design desiderata that motivates the development of MEMO.

3.1 Categorization of Activation Tensors

In long-context LLM training, the primary memory consumption originates from activation tensors, which are the outputs of computing operators in LLMs. According to their life cycles, we can categorize activations generated during the forward propagation into two classes, which are the skeletal activations and the transient activations, where the former is necessary for the backward propagation while the latter is not.

For illustration, in Figure 3(b), tensors 13, 14, 17, 18, and 19 are produced during the forward pass of a transformer layer, and are discarded before the completion of this layer's forward pass. Similarly, tensors 20, 21, 22, 23, and 24 are generated during the backward pass of this layer, and are discarded after corresponding computation. We term them "transient tensors" because they are created and discarded within a single layer's forward or backward pass. Transient tensors usually serve as temporary results. Conversely, tensors 15 and 16 are generated during the forward propagation and are needed for backward propagation, so they are discarded in this layer's backward pass. We refer to these tensors as "skeletal tensors" because they are produced during the forward pass, and are essential for the gradient calculation during the backward pass.

3.2 Analysis of Skeletal Activations

Figure 4 presents all skeletal tensors generated within a transformer layer's forward propagation, along with their sizes. We can see that the total size of all skeletal activations in a single transformer layer amounts to $16bsh$. To exemplify, when training the GPT-7B model ($h = 4096$, 32 layers) with

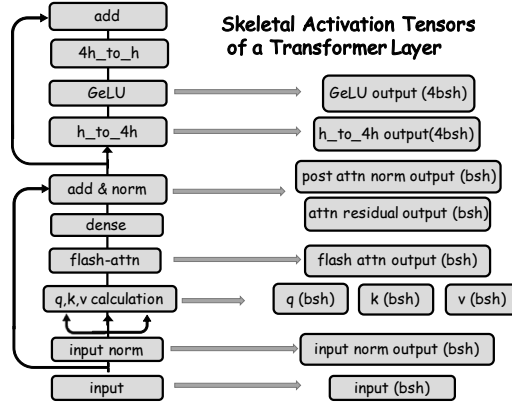


Fig. 4. Illustration of the transformer layer architecture. The sizes of skeletal activations are provided in the brackets.

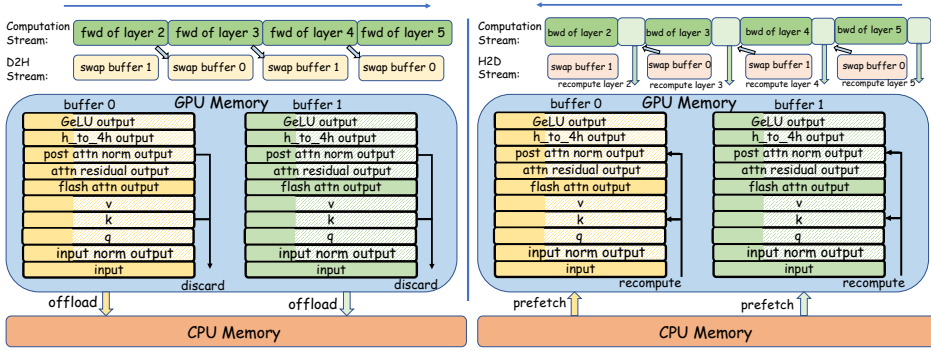


Fig. 5. Forward and backward propagation with rounding buffers for token-wise recomputation/swapping. During forward propagation, the darker part in the rounding buffers is offloaded to CPU, while the lighter part is discarded; during backward propagation, the darker part in the rounding buffers is prefetched from CPU, while the lighter part is recomputed.

a sequence length (s) of 1 million, if we store the skeletal activations in half-precision floating numbers, it would take 4096 GB for only one sequence ($b = 1$), exceeding the memory capacity of even 50 A100/H100 GPUs.

An important characteristic of skeletal activations is that they are needed by backward computation, so they must reside in GPU at least before the backward propagation of the corresponding transformer layer begins. However, maintaining all skeletal activations for backward propagation is infeasible. To this end, memory-saving techniques like recomputation and swapping, which are also prevalent in traditional data management problems [30, 64, 91], become necessary for long context training.

In LLM training, these techniques first release the skeletal activations of a transformer layer in the forward propagation, and later rematerialize them before the corresponding backward propagation.

Unfortunately, naïvely applying activation recomputation or swapping is insufficient to tackle the challenge of managing large-scale skeletal activations. Both techniques trade time for memory – the activation recomputation technique incurs extra computation overhead while the swapping

technique necessitates transmitting the activations from CPU memory to GPU memory. Using recomputation alone incurs significant additional computation overhead, while employing swapping alone can lead to CPU OOM error (when the sequence is too long) or block GPU computation (when the swapping time cannot be fully overlapped). As a result, we desiderate a meticulous orchestration of the two memory-saving techniques to manage the skeletal activations, so that we can minimize the extra overhead while accommodating the huge memory requirement in long context training of LLMs. To achieve this, we develop a *token-wise activation recomputation and swapping* mechanism, which will be demonstrated in Section 4.1.

3.3 Analysis of Transient Activations

Transient activations are intermediate results generated and discarded during the forward (or backward) pass of a transformer layer. Actually, there are more transient activations than skeletal activations in a transformer layer. Specifically, we observe that the number of transient activations can exceed 5 times that of skeletal activations. Without careful management, the frequent allocation and deallocation can lead to memory fragmentation, which degrades system performance and poses a major concern for managing massive data storage. There have been studies in the data management community focusing on memory defragmentation [16, 59], leveraging the characteristics of target workloads to devise appropriate and innovative methods. Following the methodology, we analyze the LLM training process and attempt to defragment the tensor memory. During training, memory requests are identical across both transformer layers and training iterations, providing an opportunity to manage and reuse these memory regions effectively to minimize fragmentation. In particular, the memory addresses of a single transformer layer’s transient activation tensors can be reused by all other transformer layer’s corresponding transient activation tensors. However, in practice, memory reuse is not fulfilled because the PyTorch caching allocator lack prior information of the memory request sequence during training iterations. This inspires us to statically plan the memory addresses of each transformer layer’s transient tensors, which will be described in detail in Section 4.2.

4 MEMO Design

In this section, we propose MEMO for fine-grained activation memory management. Our proposed method leverages fine-grained and structured activation management, akin to concise memos that share vital information. The main challenge of long context training is the large activation size which scales linearly w.r.t. sequence length. We propose token-wise activation recomputation and swapping, along with a bi-level memory planning to address the issue, which targets skeletal activations and transient activations, respectively. The overview of MEMO is depicted in Figure 2.

4.1 Token-wise Recomputation and Swapping

Skeletal tensors, generated during the forward pass of a transformer layer, must reside in GPU memory for the subsequent backward propagation. In practice, as sequence length grows, the size of skeletal activations increases linearly, which can easily exceed the capacity of GPU memory. As introduced in Section 2.2, currently the most widely-used technique to tackle this issue is activation recomputation, which stores only the input of each transformer layer, and discards the rest skeletal activation tensors of this layer. Prior to backward propagation of each layer, an additional forward pass of the layer is conducted to reconstruct all skeletal tensors so that the backward computation can be carried out. However, we note that the vanilla activation recomputation strategy is not an optimal choice to handle the challenge of linearly increasing skeletal activation memory, considering the following two reasons: (1) activation recomputation introduces redundant computation, thus diminishing training efficiency; and (2) the memory overhead of retaining the input tensor of each

transformer layer can still be expensive, especially when the sequence length is too long or the number of layers is too large. Take the training of GPT-7B with a context length of 1 million as an example again. For only one sequence, the input tensors of all 32 transformer layers together consume 128GB. Even using a SP degree of 8, it takes 16GB for each GPU to store the input tensors of all 32 transformer layers, which already takes up to 20% of total GPU memory capacity.

As explained in Observation 1, the computation complexity of FlashAttention w.r.t. sequence length is $O(s^2)$, while the size of skeletal activations within a transformer layer scales linearly with sequence length. This provides us with the opportunity to offload skeletal activations to CPU memory, thereby saving GPU memory. We can prefetch them back to GPU before the backward propagation of the corresponding transformer layer. The swapping of skeletal activations can overlap with GPU computations in long context training, since the CPU-GPU data transmission does not consume GPU computation units.

To facilitate the overlapping, we utilize two rounding GPU buffers to store the skeletal activations for all transformer layers. The two rounding buffers are allocated before the actual training iterations begin. As shown in Figure 5, transformer layers with even layer indices place their skeletal activation tensors in rounding buffer 0, while layers with odd layer indices use rounding buffer 1.

After the computation of transformer layer i , rounding buffer $(i\%2)$ will be offloaded to CPU using a separate CUDA stream. This happens simultaneously with the computation of transformer layer $(i+1)$. Before the forward computation of transformer layer $(i+2)$, a CUDA event is employed to ensure the content of rounding buffer $(i\%2)$ has been fully offloaded to CPU memory, thus the transformer layer $(i+2)$ can safely rewrite rounding buffer $(i\%2)$.

For backward propagation, after the backward pass of transformer layer $(i+2)$ ends, the contents within rounding buffer $(i\%2)$ become useless, and we start prefetching the skeletal activations of transformer layer i to rounding buffer $(i\%2)$ using another CUDA stream. The prefetching of transformer layer i 's skeletal activations happens simultaneously with the backward propagation of transformer layer $(i+1)$. When the sequence length is sufficiently long, with careful computation-transmission overlapping and synchronization, CPU swapping can substitute activation recomputation without incurring additional overhead.

However, there are two constraints that prevent us from offloading all skeletal activations to CPU memory.

- For sequence lengths that are not sufficiently long, the time required to offload all skeletal activations to CPU memory surpasses the computation time for a single transformer layer. This discrepancy forces the computation of transformer layer $(i+2)$ to be delayed until the offloading of rounding buffer to CPU memory is completed, thereby blocks the normal GPU computation workflow. For instance, as illustrated in Figure 1(b), when training a 7B GPT model on 8 GPUs with a TP degree of 8, ideal overlap between the computation of a transformer layer and the offloading of its skeletal activations occurs only for sequence lengths exceeding 192K. In practice, the sequence lengths of most training datasets are moderate and may be not sufficient to ensure an ideal overlap between computation and transmission.
- In theory, a longer sequence length provides more opportunities for overlapping swapping with GPU computation. However, in practice, the CPU memory is often limited. For a typical GPU server which has several terabytes CPU memory (e.g. 2TB in our environment), it is insufficient to store all skeletal activations when the sequence length is excessively long or the number of transformer layers is too large. For instance, when training the 7B model on a server equipped with 8 GPUs using a sequence length of 1 million, the skeletal activations amount to a total size of 4096GB, which is double the capacity of CPU memory.

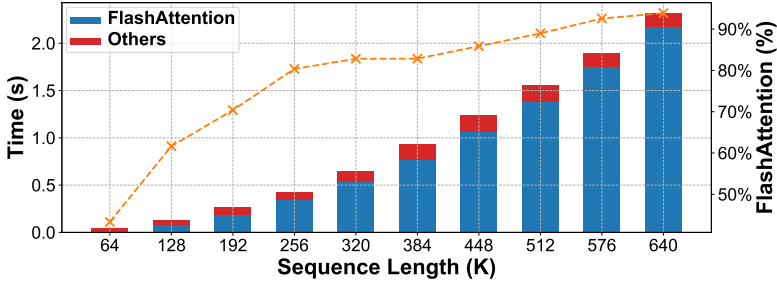


Fig. 6. Forward time of FlashAttention and other parts of a transformer layer when training a 7B GPT on 8 GPUs with a TP degree of 8.

Therefore, instead of simply offloading all skeletal activations to CPU memory, we employ selective activation swapping to ensure perfect overlap of computation and transmission for short sequences as well as to avoid depleting CPU memory for extremely long context lengths. MEMO determines the selection of swapping at both the tensor and token granularities, as depicted in Figure 5.

At the tensor granularity, we consider the benefits of leveraging the swapping technique rather than the recomputation technique of different modules. As depicted in Figure 6, FlashAttention constitutes the most substantial portion of the forward computation of a transformer layer. Notably, when the sequence length exceeds 576K, FlashAttention accounts for more than 90% of the computation involved in a single transformer layer. However, as illustrated in Figure 4, the output of FlashAttention only accounts for 6.25% of total skeletal activation size. This inspires us to offload the entire output tensor of FlashAttention to CPU memory since recomputing its output is very time-consuming. Besides, since LLMs have a layered structure, in order to reconstruct the “input_norm”, “q”, “k”, “v” tensors, we also store the input of each transformer layer to CPU, following common recomputation strategy [8].

At the token granularity, we develop the token-wise activation recomputation and swapping technique to reduce the memory consumption of all skeletal activation tensors other than the output of FlashAttention and the input of each layer. To be specific, as shown in Figure 5, for each of these skeletal activation tensors, we offload a fraction (denoted as α) to CPU, while the remaining part is discarded, ensuring perfect overlapping and to avoid CPU OOM error. Before the backward pass, the discarded part is rematerialized via recomputation while the offloaded part is prefetched.

To determine the fraction α , we solve the following problem:

$$\begin{aligned}
 \max \quad & \alpha, \\
 \text{s.t.} \quad & (S_{\text{input}} + S_{\text{attn}} + \alpha \cdot S_{\text{others}})/B \leq T_{\text{layer}}, \\
 & (n-2)(S_{\text{input}} + S_{\text{attn}} + \alpha \cdot S_{\text{others}}) \leq M_{\text{CPU}}.
 \end{aligned}$$

where S_{input} , S_{attn} , and S_{other} stand for the size of input tensor, the size of FlashAttention output tensor, the total size of other skeletal activation tensors, respectively; B is the PCIe bandwidth between GPU and CPU, T_{layer} is the forward time of a single transformer layer, n is the total number of transformer layers, and M_{CPU} stands for the capacity of CPU memory. It is worth noting that, the last two transformer layers can initiate the backward pass immediately after the forward pass, obviating the need for swapping. These variables can be easily obtained through profiling before training, so we can determine an appropriate α without much effort.

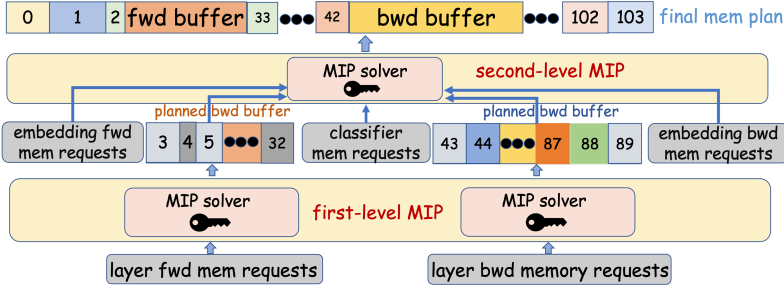


Fig. 7. Bi-level MIP algorithm.

4.2 Bi-level Memory Planning

In the previous subsection, we have tackled the management of skeletal activations by the fine-grained recomputation and swapping technique. However, frequent allocation and deallocation of the transient activation tensors still lead to GPU memory fragmentation, which forces the allocator to frequently reorganize GPU memory using time-consuming “cudaFree” and “cudaMalloc” operations. To address the issue, and to achieve full reuse of GPU memory across all transformer layers, we design a bi-level Mixed Integer Programming (MIP) method.

In practice, our initial step involves profiling the sequence of memory requests during a single training iteration. Given the memory request sequence, the challenge lies in determining the address of each requested tensor while at the same time minimizing the peak memory usage. This task aligns with the well-established offline Dynamic Storage Allocation (DSA) problem [76], which can be formulated as a Mixed Integer Programming (MIP) problem. A concise overview of this formulation is shown as follows.

The offline DSA problem handles a sequence of memory allocations and deallocations, and aims to determine the address of each allocated memory block and at the same time minimizing the peak memory usage. Parameters of offline DSA problem includes:

- n , the number of requested tensors.
- S_i , the size of requested tensor i , for $\forall i \in \{1, 2, \dots, n\}$.
- $E = \{(i, j) | \text{tensor } i, j \text{ have overlapped lifespan}\}$.

And the problem can be written as

$$\begin{aligned} \min \quad & M, \\ \text{s.t.} \quad & \begin{cases} A_i + S_i \leq M, i \in \{1, 2, \dots, n\}, \\ A_i + S_i \leq A_j + z_{ij} \cdot M_{cap}, (i, j) \in E, \\ A_j + S_j \leq A_i + (1 - z_{ij}) \cdot M_{cap}, (i, j) \in E, \\ 0 \leq M \leq M_{cap}, \\ A_i \geq 0, i \in \{1, 2, \dots, n\}, \end{cases} \end{aligned}$$

where A_i stands for the address of requested tensor i , M stands for the peak memory usage, M_{cap} is the memory capacity, and z_{ij} is defined as

$$z_{ij} = \begin{cases} 0, & A_i + S_i \leq A_j, (i, j) \in E, \\ 1, & A_j + S_j \leq A_i, (i, j) \in E. \end{cases}$$

Here the first constraint and the last two constraints define and limit peak memory, while the second and third constraints ensure non-overlapping tensors. Following this formulation, the

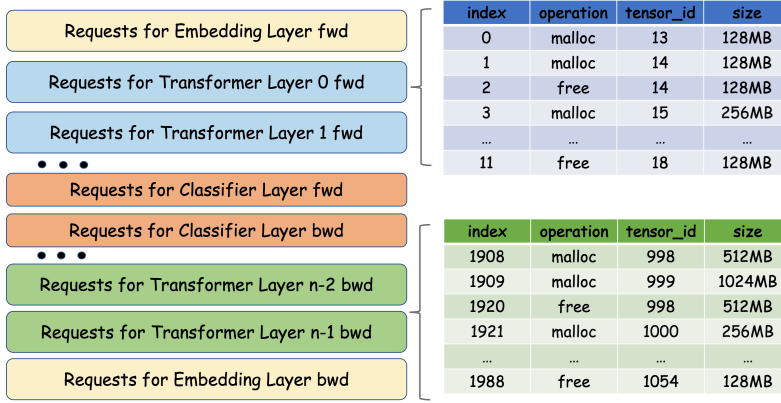


Fig. 8. Memory request sequence during training.

solution for each tensor's address is optimal. However, modern LLM training involves thousands of allocation and deallocation requests within a single training iteration, which makes this NP-hard MIP problem computationally intractable. Consequently, it's infeasible to solve this MIP problem in one pass, given the prohibitively high time cost.

Fortunately, all transformer layers have identical structures and memory request sequences, which presents repetitive substructures within the MIP problem. By leveraging this inherent repetitiveness, we can instead devise a bi-level hierarchical MIP optimization algorithm, which is both computationally feasible and effective.

As discussed in Section 2.1.1, a typical LLM consists of an embedding layer, n consecutive transformer layers, and a final classification layer. As shown in Figure 8, each layer has forward memory request sequence and backward memory request sequence. The memory request sequence is in the form of a sequence of "malloc tensor_id size" and "free tensor_id size". Since all transformer layers in an LLM are identical, they have the same forward/backward pass memory request sequence.

As shown in the bottom of Figure 7, we first solve the offline DSA sub-problem for just one transformer layer's forward (backward) pass, which is called the first-level MIP. The scale of the first-level MIP is small enough to tackle. This offline DSA problem can be simply solved by any MIP solver (e.g. Gurobi [24]). After this step, the peak memory needed for the forward (backward) propagation of a single transformer layer, as well as the address of each transient tensor within a transformer layer is determined. After solving the sub-problem for one transformer layer, all other transformer layers can reuse the same memory address for (de)allocation.

Besides identical transformer layers, LLMs also have other layers that process input tokens and classify output tokens. Therefore, after solving the first MIP for one-layer DSA problem, we conduct the second MIP for the whole LLM training to generate the peak memory requirement and addresses of all transient activation tensors. To simplify the optimization process, we can replace the original fine-grained memory request sequence of each transformer layer's forward (backward) propagation with a "pseudo" large memory request pair, as shown in Figure 7. The size of this "pseudo" memory block corresponds to the memory usage of each transformer layer as determined by the first-level MIP. After the substitution, this reformulated memory request sequence also satisfies the formulation of an offline DSA problem, with a size small enough to be efficiently solved. We then leverage the MIP solver again to solve this second-level MIP problem.

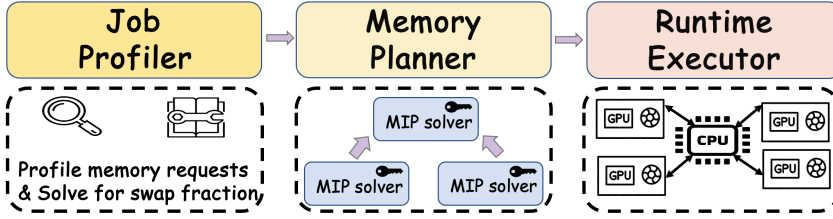


Fig. 9. Overall architecture of MEMO.

After this step, the addresses of all activation tensors, and the peak memory needed for all transient activation tensors can be determined.

4.3 System Implementation

4.3.1 Overview. Figure 9 illustrates the overall architecture of MEMO. First, the job profiler takes in the model configuration, then executes a training iteration to profile the memory requests directed to the PyTorch CUDA allocator during the training phase. The job profiler also determines offloading fraction α by solving optimization problem in Section 4.1. These memory requests comprise a sequence of allocation and deallocation instructions. Afterwards, the memory planner receives the memory requests, executes the bi-level MIP optimization algorithm and, generates a memory plan, which constitutes the addresses of all transient activation tensors during one training iteration. Finally, the runtime executor reads the memory plan and conducts the training process.

4.3.2 Job Profiler. The job profiler is designed to profile the memory request sequence during a training iteration. To implement the module, we have extended the PyTorch CUDA allocator with extra interfaces that log each memory request it receives, in the format of “malloc tenosr_id size” and “free tensor_id size”.

However, naively recording all memory requests may lead to OOM error. For example, directly profiling a GPT-7B model with a sequence length of 512K on 8 GPUs can result in OOM error. Fortunately, all transformer layers have identical memory footprint. We leverage this property by only profiling one transformer layer’s memory footprint and then applying it to all transformer layers.

When the sequence is too long, we cannot even profile one single transformer layer. In such extreme cases, we turn to the CUDA Unified Memory feature, which enables the swapping between GPU memory and CPU memory under the hook, creating an illusion of unlimited GPU memory. By integrating CUDA Unified Memory support into the PyTorch CUDA allocator, we have successfully managed to profile the training of extremely long context lengths.

The profiler also gathers the basic information to determine α in Section 4.1, including the size of each skeletal activation tensor, and the forward time of a one layer. Subsequently, it solves for the optimal α to maximize the overlapping of computation and transmission as well as to avoid CPU OOM error.

4.3.3 Memory Planner. Given the memory request sequence generated by the job profiler, memory planner executes the bi-level MIP optimization algorithm as introduced in Section 4.2 to generate a memory plan, which includes the address of each transient activation tensor and the peak memory usage needed during training. MEMO uses the Gurobi [24] optimizer to solve the MIP problems. In all our experiments, memory planning takes less than 5 minutes, which is negligible compared to the training time of LLMs.

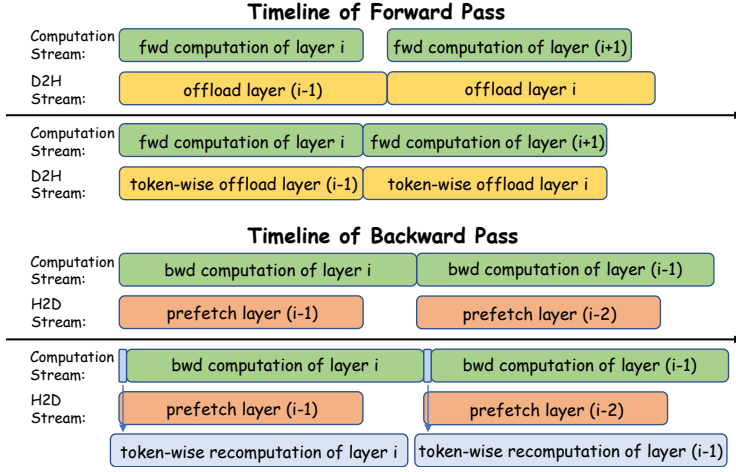


Fig. 10. Scheduling of computation, offloading and prefetching w/ and w/o token-wise recomputation. Given the superior computing ability of modern GPUs, the recomputation part is faster than the offloading part that blocks forward computation.

Table 2. Configurations of the evaluated models.

Model Size	Hyper Parameters				
	n_{layers}	h	h_{ffn}	n_{head}	n_{vocab}
7B	32	4096	16384	32	50257
13B	40	5120	20480	40	50257
30B	48	7168	28672	56	50257
65B	80	8192	32768	64	50257

4.3.4 Runtime Executor. The runtime executor takes the memory plan, and executes the training process. It is built on the top of Megatron-LM [77] with TransformerEngine [56], one of the most popular training frameworks. The runtime executor utilizes two rounding buffers for the storage of skeletal activations, as introduced in Section 4.1. Meanwhile, the transient activation tensors are (de)allocated according to the memory plan.

Three CUDA streams are employed for efficient overlapping of data transmission and GPU computation, which are for GPU computation, activation offloading from GPU to CPU, and activation prefetching from CPU to GPU, respectively. Figure 10 shows the scheduling of computation and transmission. After the computation of one layer’s forward pass, the skeletal activations of this layer are scheduled to be transferred to the CPU memory, which can overlap with the computation of the next layer. Before the backward computation of one layer, the forward skeletal activations of the previous layer are scheduled to be fetched back to GPU. In addition, token-wise tensor recomputation is scheduled before the layer’s backward pass. By hiding the activation swapping with computation and enabling the lightweight, token-wise activation recomputation, MEMO minimizes the overhead of activation rematerialization at full stretch.

5 Experiments

In this section, we conduct experiments across various model sizes and input sequence lengths to show that MEMO achieves superior efficiency in long context training of LLMs.

5.1 Setup

Hardware: Our experiments are conducted on an A800 GPU cluster, with each node equipped with 8 NVIDIA A800 GPUs (80GB). The GPUs within each node are interconnected via NVLinks (400GB/s), while the nodes are interconnected through Infiniband (200GB/s). Each node has 2TB CPU memory, and the GPU-CPU communication bandwidth is 32GB/s.

Baselines: We select two widely-used LLM training frameworks as baselines. The first is Megatron-LM (commit id: ccfeda47cb) [77] in conjunction with TransformerEngine (v1.3) [56]. Megatron-LM, maintained by NVIDIA, is renowned for its comprehensive support of hybrid parallelisms, including DP, TP, PP, SP, and CP. The other baseline is Megatron-Deepspeed (commit id 7eb36a11b3) paired with DeepSpeed (v0.14.3) [69], which is recognized for ZeRO optimizers and DeepSpeed-Ulysses [28], a novel parallel training strategy designed for long context LLM training. For all baselines, we employ FlashAttention [11, 12] and mixed-precision training [31, 49], which are commonly used in LLM training.

Metrics: We use three important evaluation metrics to measure the training efficiency: Model FLOPs Utilization (MFU), Tokens per GPU per Second (TGS), and wall-clock time. MFU is defined as the ratio of model FLOPs per second to the theoretical peak FLOPs per second of the GPU (e.g. 312 TFLOPS for NVIDIA A800 GPUs) [35]. Based on FlashAttention [11, 12] and considering the causal mask, the exact formula for calculating model FLOPS per sample is:

$$6 \cdot s \cdot P + 6 \cdot n \cdot h \cdot s^2.$$

MFU is a standard metric that measures the training efficiency of how model FLOPs utilize computational resources. On the other hand, TGS directly measures training throughput, and wall-clock time reflects the latency of training each batch, providing a clear view of how quickly a model can be trained using a given amount of training samples. All metrics are crucial for LLM researchers and engineers, enabling comparisons among various training strategies (including distributed parallelisms and activation recomputation).

Workloads: Our experiments cover a wide range of workloads to examine the strength of MEMO. In particular, we consider training the 7B, 13B, 30B and 65B GPT models on 8, 16, 32, and 64 GPUs, respectively, with various sequence lengths ranging from 4K to 1408K. The detailed model configurations are shown in Table 2.

5.2 End-to-end Evaluation

We compare the end-to-end training efficiency of MEMO and two baselines. Table 3 shows the MFU, TGS, and wall-clock time of DeepSpeed-Ulysses, Megatron-LM and MEMO under different training workloads. During evaluation, we manually adjust the distributed parallelism strategies for each system and each workload to achieve optimal training performance for fair comparisons.

Overall, MEMO is capable of training longer sequences than the competitors. Across the training of 7B, 13B, 30B, and 65B models on 8, 16, 32, and 64 GPUs, DeepSpeed-Ulysses supports sequence lengths of 256K, 256K, 128K, and 1280K, while Megatron-LM supports sequence lengths of 640K, 640K, 384K, 512K. In comparison, MEMO achieves superior performance in all scenarios, enabling training sequence lengths of 1024K, 1408K, 1280K, and 1408K. Megatron-LM only supports up to 640K sequence length, even if we have leveraged a high model parallel degree and enabled the memory reduction techniques. This is unsurprising since it overlooks the memory fragmentation issue, leading to OOM for large sequence lengths. DeepSpeed, thanks to its support of DeepSpeed-Ulysses sequence parallel and ZeRO-3 optimizer, is capable of training 1280K sequence length when training the 65B model on 64 GPUs. When training smaller models, DeepSpeed supports only very small sequence lengths. This is because it can only utilize a small SP degree of 8, which either aligns to the number of GPUs or is dividable by the number of attention heads (40 and 56).

Table 3. MFU (Model FLOPS Utilization), TGS (Tokens per GPU per Second), and wall-clock time of different methods on different numbers of GPUs, model sizes, and sequence lengths. “DS” stands for “DeepSpeed”, and “Mega” stands for “Megatron-LM”. $\times_{oom}$ stands for Out Of Memory error when the system runs out of GPU memory, and $\times_{oohm}$ stands for Out Of Host Memory error, which means the CPU memory is depleted. “h”, “m” and “s” represent hour, minute and second respectively.

SeqLen	7B GPT on 8 GPUs			13B GPT on 16 GPUs			30B GPT on 32 GPUs			65B GPT on 64 GPUs		
	DS	Mega	MEMO	DS	Mega	MEMO	DS	Mega	MEMO	DS	Mega	MEMO
4K	44.71%	49.45%	49.43%	44.08%	46.17%	46.17%	43.70%	50.67%	50.67%	43.89%	53.61%	53.61%
	3235.64	3578.86	3578.86	1675.53	1755.23	1755.23	724.44	840.14	840.14	337.95	412.90	412.90
	2.53s	2.29s	2.29s	2.44s	2.33s	2.33s	2.83s	2.44s	2.44s	3.03s	2.48s	2.48s
8K	46.28%	51.57%	51.57%	38.90%	50.91%	50.91%	38.78%	49.27%	49.27%	43.34%	53.27%	53.27%
	3116.25	3472.95	3472.95	1393.24	1823.52	1823.52	615.22	781.68	781.68	321.00	394.61	394.61
	5.26s	4.72s	4.72s	5.88s	4.50s	4.50s	6.66s	5.24s	5.24s	6.38s	5.19s	5.19s
16K	36.41%	51.29%	51.29%	36.95%	51.52%	51.52%	34.54%	49.84%	49.84%	40.98%	51.10%	51.10%
	2152.65	3032.14	3032.14	1186.24	1654.11	1654.11	504.64	728.22	728.22	282.05	351.58	351.58
	15.2s	10.8s	10.8s	13.8s	9.91s	9.91s	16.2s	11.2s	11.2s	14.5s	11.6s	11.6s
32K	35.91%	52.75%	52.75%	34.51%	49.79%	49.79%	35.19%	48.06%	48.06%	35.73%	34.00%	44.79%
	1706.13	2506.39	2506.39	917.96	1324.27	1324.27	443.77	606.16	606.16	215.32	204.83	269.84
	38.4s	26.1s	26.1s	35.7s	24.7s	24.7s	36.9s	27.0s	27.0s	38.0s	39.9s	30.4s
64K	27.95%	41.55%	52.34%	27.97%	38.51%	52.65%	29.93%	35.76%	52.12%	31.05%	22.79%	47.80%
	935.76	1417.74	1786.22	553.85	762.46	1042.50	296.41	354.20	516.16	149.80	109.94	230.62
	2m17s	1m33s	1m13s	1m58s	1m26s	1m3s	1m51s	1m33s	1m3s	1m49s	2m19s	1m11s
128K	25.46%	24.13%	50.96%	25.45%	23.02%	50.93%	25.54%	14.70%	49.66%	26.13%	15.10%	48.61%
	555.51	526.62	1111.99	333.51	301.67	667.41	176.92	101.85	344.09	90.15	52.10	167.69
	7m52s	8m18s	3m56s	6m33s	7m15s	3m16s	6m10s	10m43s	3m10s	6m4s	10m30s	3m15s
256K	23.38%	29.07%	53.62%	21.98%	25.30%	51.22%	$\times_{oom}$	17.15%	50.00%	22.07%	9.57%	49.87%
	296.48	368.65	679.92	171.82	197.78	400.39	$\times_{oom}$	72.24	216.41	48.51	21.03	109.58
	29m28s	23m42s	12m51s	25m26s	22m5s	10m55s	$\times_{oom}$	29m30s	10m6s	22m31s	51m56s	9m58s
384K	$\times_{oom}$	27.98%	53.04%	$\times_{oom}$	22.88%	51.91%	$\times_{oom}$	23.32%	50.69%	20.40%	12.07%	48.85%
	$\times_{oom}$	250.07	474.02	$\times_{oom}$	127.40	289.11	$\times_{oom}$	73.37	159.52	32.89	19.46	78.76
	$\times_{oom}$	52m25s	27m39s	$\times_{oom}$	51m26s	22m40s	$\times_{oom}$	44m39s	20m32s	49m49s	1h24m	20m48s
512K	$\times_{oom}$	34.43%	51.84%	$\times_{oom}$	29.10%	52.40%	$\times_{oom}$	$\times_{oom}$	51.06%	19.83%	5.32%	49.71%
	$\times_{oom}$	237.56	357.70	$\times_{oom}$	125.86	226.65	$\times_{oom}$	$\times_{oom}$	126.23	25.24	6.77	63.29
	$\times_{oom}$	1h14m	48m51s	$\times_{oom}$	1h10m	38m33s	$\times_{oom}$	$\times_{oom}$	34m37s	1h27m	5h23m	34m31s
640K	$\times_{oom}$	30.90%	52.59%	$\times_{oom}$	19.41%	52.13%	$\times_{oom}$	$\times_{oom}$	51.72%	19.06%	$\times_{oom}$	50.05%
	$\times_{oom}$	173.63	295.51	$\times_{oom}$	83.94	184.33	$\times_{oom}$	$\times_{oom}$	105.28	20.04	$\times_{oom}$	52.65
	$\times_{oom}$	2h6m	1h14m	$\times_{oom}$	2h10s	59m15s	$\times_{oom}$	$\times_{oom}$	51m52s	2h16m	$\times_{oom}$	51m52s
768K	$\times_{oom}$	$\times_{oom}$	51.89%	$\times_{oom}$	$\times_{oom}$	51.71%	$\times_{oom}$	$\times_{oom}$	51.18%	19.53%	$\times_{oom}$	51.16%
	$\times_{oom}$	$\times_{oom}$	245.76	$\times_{oom}$	$\times_{oom}$	154.63	$\times_{oom}$	$\times_{oom}$	88.55	17.50	$\times_{oom}$	45.84
	$\times_{oom}$	$\times_{oom}$	1h47m	$\times_{oom}$	$\times_{oom}$	1h25m	$\times_{oom}$	$\times_{oom}$	1h14m	3h7m	$\times_{oom}$	1h11m
896K	$\times_{oom}$	$\times_{oom}$	52.71%	$\times_{oom}$	$\times_{oom}$	51.76%	$\times_{oom}$	$\times_{oom}$	51.50%	19.12%	$\times_{oom}$	51.05%
	$\times_{oom}$	$\times_{oom}$	251.98	$\times_{oom}$	$\times_{oom}$	134.08	$\times_{oom}$	$\times_{oom}$	77.48	14.93	$\times_{oom}$	39.85
	$\times_{oom}$	$\times_{oom}$	2h22m	$\times_{oom}$	$\times_{oom}$	1h54m	$\times_{oom}$	$\times_{oom}$	1h39m	4h16m	$\times_{oom}$	1h36m
1024K	$\times_{oom}$	$\times_{oom}$	52.30%	$\times_{oom}$	$\times_{oom}$	52.06%	$\times_{oom}$	$\times_{oom}$	51.24%	19.00%	$\times_{oom}$	51.27%
	$\times_{oom}$	$\times_{oom}$	188.73	$\times_{oom}$	$\times_{oom}$	118.96	$\times_{oom}$	$\times_{oom}$	68.20	13.14	$\times_{oom}$	35.45
	$\times_{oom}$	$\times_{oom}$	3h5m	$\times_{oom}$	$\times_{oom}$	2h27m	$\times_{oom}$	$\times_{oom}$	2h8m	5h33m	$\times_{oom}$	2h3m
1152K	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	51.74%	$\times_{oom}$	$\times_{oom}$	51.73%	19.11%	$\times_{oom}$	51.20%
	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	105.74	$\times_{oom}$	$\times_{oom}$	61.72	11.86	$\times_{oom}$	31.77
	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	3h6m	$\times_{oom}$	$\times_{oom}$	2h39m	6h43m	$\times_{oom}$	2h35m
1280K	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	51.78%	$\times_{oom}$	$\times_{oom}$	51.59%	18.90%	$\times_{oom}$	51.42%
	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	95.72	$\times_{oom}$	$\times_{oom}$	55.79	10.64	$\times_{oom}$	28.94
	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	3h48m	$\times_{oom}$	$\times_{oom}$	3h16m	8h33m	$\times_{oom}$	3h9m
1408K	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	52.10%	$\times_{oom}$	$\times_{oom}$	$\times_{oohm}$	$\times_{oom}$	$\times_{oom}$	51.45%
	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	87.93	$\times_{oom}$	$\times_{oom}$	$\times_{oohm}$	$\times_{oom}$	$\times_{oom}$	26.50
	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	4h33m	$\times_{oom}$	$\times_{oom}$	$\times_{oohm}$	$\times_{oom}$	$\times_{oom}$	3h47m

In contrast, by token-wise recomputation/swapping and memory planning, MEMO is able to train sequences over 1 million tokens in all scenarios.

Table 4. MFU of different methods for ablation studies. The experiments are conducted by training the 7B model on 8 GPUs.

Method	Sequence Length							
	64K	128K	256K	384K	512K	640K	768K	896K
Full Recomputation	41.19%	23.00%	29.07%	25.67%	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$	$\times_{oom}$
Full Recomputation + Memory Plan	42.91%	43.17%	42.05%	42.49%	41.90%	42.15%	$\times_{oom}$	$\times_{oom}$
Full Swapping + Memory Plan	37.40%	46.33%	53.62%	$\times_{oohm}$	$\times_{oohm}$	$\times_{oohm}$	$\times_{oohm}$	$\times_{oohm}$
MEMO (Fine-grained Management + Memory Plan)	47.99%	50.96%	53.62%	53.04%	51.84%	52.59%	51.89%	52.71%

Table 5. Impact of offloading fraction α on training efficiency.

Sequence Length	Value of α								
	0.000	0.125	0.250	0.375	0.500	0.625	0.750	0.875	1.000
192K	51.04%	51.86%	52.14%	52.30%	52.72%	52.78%	53.11%	51.74%	49.69%
256K	52.03%	52.05%	52.19%	52.45%	52.70%	52.96%	53.21%	53.29%	53.62%
320K	52.15%	52.36%	52.52%	52.66%	52.92%	53.10%	53.35%	$\times_{oohm}$	$\times_{oohm}$
384K	52.09%	52.31%	52.45%	52.58%	53.04%	$\times_{oohm}$	$\times_{oohm}$	$\times_{oohm}$	$\times_{oohm}$

When training with relatively short sequences ranging from 4K to 32K, there's little memory pressure. In such cases, activation recomputation is often unnecessary, and different methods may use the same parallelism strategies. Consequently, MEMO falls back to Megatron-LM, resulting in the same performance. Nevertheless, MEMO significantly outperforms the baselines when the sequence length reaches 64K or even higher.

Furthermore, when comparing MEMO to the baselines with aligned sequence lengths, MEMO achieves superior MFU, TGS, and wall-clock time. Across all experimented workloads, MEMO achieves an average MFU of 51.04%. In contrast, Megatron-LM and DeepSpeed only achieve an average MFU of 35.01% and 30.74%, respectively. On average, MEMO achieves 1.97 $\times$ and 1.80 $\times$ MFU compared to Megatron-LM and DeepSpeed, respectively.

The deficiencies of baselines are not surprising — due to the unsatisfactory memory management, they are forced to utilize less-efficient configurations of parallelism strategies and rematerialization options.⁶ Firstly, the baselines need to employ a high model parallelism degree (i.e. large TP degree and/or large SP degree) to avoid OOM error. For instance, when training the 65B model with 256K sequence length, Megatron-LM has to use a TP degree of 16, while MEMO can support a TP degree of 8. Given the much lower inter-node bandwidth compared to intra-node, the TP communication cost of MEMO is 63% lower than that of Megatron-LM (44.3 v.s. 120.4 seconds). Secondly, the baselines require full activation recomputation to avoid the OOM error, leading to over 20% redundant GPU calculation. In contrast, MEMO integrates fine-grained recomputation and swapping, incurring smaller overhead. Last but not least, the lack of memory planning often triggers memory reorganization during training, which blocks GPU computation and significantly hampers training efficiency. When training the 7B model on 8 GPUs using Megatron-LM, the memory reorganization operation is triggered 6 times and 16 times per iteration for sequence lengths of 128K and 256K, respectively. A detailed ablation study is presented in Section 5.3.

As a result, MEMO consistently achieves an MFU of approximately 50% across all model sizes and sequence lengths, enabling more efficient training of significantly long sequences compared to the baselines.

⁶We have provided the detailed configurations in our supplementary materials.

5.3 Ablation Studies

Next, we assess the effectiveness of the proposed techniques in MEMO. All experiments in ablation studies are conducted by training the 7B model on 8 GPUs, keeping the parallelism configuration fixed at a TP degree of 4 and a CP degree of 2.

5.3.1 Effectiveness of Memory Planning. To evaluate the effectiveness of memory planning, we evaluate two variants of MEMO with full recomputation, both with and without memory planning. As shown in the first two rows of Table 4, without memory planning, the longest sequence supported is only 384K, achieving an MFU of 25.67%. After applying memory planning, the longest supported sequence length increases to 640K, with an MFU of 42.15%. The results are reasonable since full recomputation without memory planning has severe memory fragmentation, resulting in OOM errors in large sequence length scenarios.

Additionally, the frequent GPU memory reorganization process further impairs training efficiency. By employing memory planning, the fragmentation issue can be eliminated, providing more memory for longer context training. Getting rid of GPU memory reorganization, memory planning brings an average of $1.51\times$ MFU when facing the same context length.

5.3.2 Effectiveness of Token-wise Recomputation. For token-wise recomputation and swapping, we compare MEMO and its variants, one with full recomputation and another with full swapping. The results are shown in the last three rows of Table 4. When training with appropriate sequence length, which is 256k in this scenario, the computation time of one transformer layer can fully overlap with the offloading time of a layer's activations. Therefore, full swapping with memory planning can achieve an MFU of 53.62% under 256K sequence length, far exceeding the 42.05% achieved by full recomputation with memory planning. However, for short sequence lengths, such as 64K, the offloading time of one layer's activations blocks the GPU computation, resulting in a lower MFU than full recomputation. Full swapping presents another challenge as the sequence length grows longer: the host memory is rapidly depleted by offloaded activations, leading to OOHM errors. Full recomputation, which does not offload the input tensor of each layer, also encounters OOM errors at a sequence length of 768K.

By employing token-wise recomputation together with swapping, MEMO consistently improves training efficiency for both short and long context lengths. For short sequence lengths like 64K, our tensor-level design only offloads the input tensor of the transformer layer and the FlashAttention output tensor to CPU memory, enabling efficient overlap of GPU computation and data transmission. For long context lengths, our rounding-buffer token-level management successfully avoids depleting the CPU memory, and incurs only minimal recomputation overhead. Among all the methods, MEMO supports the longest sequence length. Considering MFU, MEMO achieves an average of $1.22\times$ MFU compared to full recomputation with memory planning, and an average of $1.13\times$ MFU compared to full offloading with memory planning.

5.3.3 Effect of the Offloading Fraction α . To investigate the impact of the offloading fraction α on training efficiency, we train a 7B model on 8 GPUs, with TP= 4, CP= 2 and varying α values. The results are presented in Table 5. For a sequence length of 192K, the MFU initially increases with α , peaking at 53.11% when $\alpha = 0.75$, and then declines. This trend occurs because, for $\alpha < 0.75$, a higher α reduces recomputation overhead by increasing the proportion of skeletal activations that are swapped. Beyond this point, however, further increases in α lead to swapping overhead that exceeds the computation time of an individual transformer layer, stalling GPU computations until the swapping concludes and thus reducing the MFU. At a sequence length of 256K, the computation time of a single transformer layer, due to the quadratic complexity of self-attention, surpasses the offloading time for all skeletal activations in this layer. Consequently, training efficiency

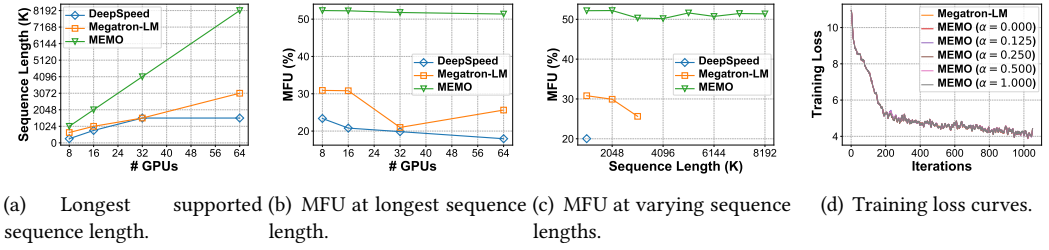


Fig. 11. (a), (b): The longest supported sequence length and corresponding MFU of DeepSpeed, Megatron-LM and MEMO when training the 7B model on various numbers of GPUs. (c): The MFU when training the 7B model on 64 GPUs with sequence length varying from 1024K to 8192K. (d): Training loss curves of Megatron-LM and MEMO with different α .

consistently improves as α increases. For longer sequences such as 320K and 384K, offloading all skeletal activations becomes unviable due to CPU memory constraints. Considering the constraints, MEMO can effectively identify the optimal α to minimize recomputation overhead.

5.4 Scalability

To show the scalability of MEMO, we train the 7B model on 8, 16, 32, and 64 GPUs respectively, and report the maximum supported sequence length. As shown in Figure 11(a), when the number of GPUs increases, the maximum sequence length supported by MEMO increases linearly. When training on 8, 16, 32, and 64 GPUs, MEMO is capable of training 1, 2, 4, 8 million sequence lengths, respectively, which demonstrates ideal scalability. MEMO also consistently maintains an MFU of over 50% across different numbers of GPUs, as shown in Figure 11(b).

For DeepSpeed, as the number of GPUs increases, it can enlarge the SP degree, leading to longer supported sequence length. Note that since the 7B model has 32 attention heads, the maximum SP degree is 32. As a result, DeepSpeed achieves the same maximum sequence length of 1536K for both 32 and 64 GPUs. Megatron-LM supports context parallelism, which has better scalability. When the number of GPUs increases, the longest sequence length it can handle grows sublinearly. Compared to the baselines, MEMO introduces fine-grained activation memory management, achieving not only ideal scalability but also better efficiency.

We also evaluate the MFU of the three systems when training the 7B model on 64 GPUs with sequence lengths varying from 1024K to 8192K. In Figure 11(c), as sequence length increases, the MFU of MEMO maintains above 50%, surpassing the baselines.

5.5 Convergence of MEMO

To confirm the correctness of our system implementation, we conduct a convergence experiment. We train a 7B model with 128K sequence length on 8 GPUs and compare the convergence of MEMO and Megatron-LM for 1000 iterations. For both systems, we fix the parallelism strategy to TP= 4 and CP= 2. For MEMO, we enumerate the value of α in $\{0, 0.125, 0.25, 0.5, 1\}$. As shown in Figure 11(d), the loss curves of MEMO with different α values all align with Megatron-LM, confirming the correctness of MEMO.

6 Related Work

Data management and machine learning: Recently, the intersection of data management and machine learning has gained popularity, with numerous studies emerging from the data

management community [19, 27, 33, 36, 39, 44–47, 50–52, 57, 75, 82, 83, 86, 88, 89, 93, 95, 97, 99, 100, 102]. Some works focus on optimizing I/O data management to accelerate data pre-processing pipelines [27, 50, 51, 82]. Another work proposes to use a fine-grained automatic parallelism strategy to speed up model training [46]. MEMO also employs data management techniques and methodologies to enhance the efficiency of machine learning tasks, specifically addressing the challenges associated with long-sequence LLM training.

Parallelism strategies for long context training: To tackle the challenge of long context training, DeepSpeed-Ulysses [28] employs an novel AllToAll communication to facilitate the partition of input sequence among GPUs, achieving lower communication overhead compared with Megatron-LM sequence parallelism [35]. LightSeq [37], Ring Attention [42], and Megatron-LM context parallelism [55] propose to split the sequence within self-attention computation, achieving better scalability. Recent efforts in the realm of sequence and context parallelisms [7, 17, 20] aim to integrate multiple strategies and enhance existing distributed settings. It is worth noting that the fine-grained memory management of MEMO is orthogonal to these distributed parallelism strategies.

Activation recomputation and swapping: Capuchin [63] proposes to combine recomputation and swapping to reduce memory footprint during training. The swapping decision is made by considering the tensor access pattern. In addition to tensor recomputation, MegTaichi [25] also proposes to co-optimize the tensor partition. Coop [96] notices that naive tensor recomputation leads to severe memory fragmentation, and proposes heuristics to reduce memory fragmentation during tensor recomputation. While these works offer solutions for common deep learning models, they do not take advantage of the specific characteristics of LLM training to achieve full overlapping and fragmentation minimization.

Memory planning for deep learning models: The memory allocation problem in deep learning models can be regarded as a DSA problem and solved by MIP [76]. OLLA [78] proposes to optimize the lifetime and memory location of tensors during the training process by solving a joint ILP problem, reducing the peak memory during training. However, it does not exploit the repetitive substructure in LLMs and relies on heuristics to simplify the integer programming problem.

7 Conclusion

In this paper, we proposed MEMO to address the memory challenges in long context LLM training. We designed a fine-grained activation recomputation and swapping strategy to fully utilize the idle PCIe bandwidth during the GPU computation, thereby reducing the activation rematerialization cost in long context LLM training. We employed a bi-level MIP technique to first solve the problem of memory allocation within one transformer layer, and then reused the same memory space for each identical layer so as to eliminate memory fragmentation. Through extensive experiments, we demonstrated that MEMO achieved an average of $1.97\times$ MFU compared to Megatron-LM. By leveraging fine-grained tensor memory management, MEMO achieved 52.30% MFU when training 7B LLM with 1 million sequence length on only 8 A800 GPUs.

Acknowledgement

This work is supported by National Science and Technology Major Project (2022ZD0116315), National Natural Science Foundation of China (U22B2037, U23B2048, 62402011), Beijing Municipal Science and Technology Project (Z231100010323002), China National Postdoctoral Program for Innovative Talents (BX20230012), China Postdoctoral Science Foundation (2024M750103), Beijing Natural Science Foundation (4244080), research grant No. IPT-2024JK29, PKU-Tencent joint research Lab, and High-performance Computing Platform of Peking University. Fangcheng Fu and Bin Cui are the corresponding authors.

References

- [1] Moonshot AI. 2024. KimiChat. <https://kimi.moonshot.cn/>
- [2] Microsoft Research AI4Science and Microsoft Azure Quantum. 2023. The Impact of Large Language Models on Scientific Discovery: a Preliminary Study using GPT-4. *arxiv preprint*, 2311.07361 (2023).
- [3] Jason Ansel, Edward Z. Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [4] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arxiv preprint*, 2004.05150 (2020).
- [5] Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. 2023. Accurate medium-range global weather forecasting with 3D neural networks. *Nat.* 619 (2023).
- [6] Abel Chandra, Laura Tünnemann, Tommy Löfstedt, and Regina Gratz. 2023. Transformer-based deep learning for predicting protein properties in the life sciences. *Elife* 12 (2023).
- [7] Qiaoling Chen, Diandian Gu, Guoteng Wang, Xun Chen, YingTong Xiong, Ting Huang, Qinghao Hu, Xin Jin, Yonggang Wen, Tianwei Zhang, and Peng Sun. 2024. InternEvo: Efficient Long-sequence Large Language Model Training via Hybrid Parallelism and Redundant Sharding. *arxiv preprint*, 2401.09149 (2024).
- [8] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. *arxiv preprint*, 1604.06174 (2016).
- [9] Alibaba Cloud. 2024. Tongyi Qianwen. <https://tongyi.aliyun.com/qianwen/>
- [10] PyTorch Contributors. 2023. Understanding CUDA Memory Usage. https://pytorch.org/docs/stable/torch_cuda_memory.html.
- [11] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [13] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, Marc'Aurelio Ranzato, Andrew W. Senior, Paul A. Tucker, Ke Yang, and Andrew Y. Ng. 2012. Large Scale Distributed Deep Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [14] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuan Yuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. 2024. LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations (ICLR)*.
- [16] Dominik Dürner, Viktor Leis, and Thomas Neumann. 2019. On the Impact of Memory Allocation on High-Performance Query Processing. In *Proceedings of the 15th International Workshop on Data Management on New Hardware (DaMoN)*.
- [17] Jiarui Fang and Shangchun Zhao. 2024. USP: A Unified Sequence Parallelism Approach for Long Context Generative AI. *arxiv preprint*, 2405.07719 (2024).
- [18] Yao Fu, Rameswar Panda, Xinyao Niu, Xiang Yue, Hannaneh Hajishirzi, Yoon Kim, and Hao Peng. 2025. Data engineering for scaling language models to 128K context. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [19] Shihong Gao, Yiming Li, Xin Zhang, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. SIMPLE: Efficient Temporal Graph Neural Network Training at Scale with Dynamic Data Placement. In *Proceedings of the ACM on Management of Data (SIGMOD)*.
- [20] Hao Ge, Fangcheng Fu, Haoyang Li, Xuanyu Wang, Sheng Lin, Yujie Wang, Xiaonan Nie, Hailin Zhang, Xupeng Miao, and Bin Cui. 2024. Enabling Parallelism Hot Switching for Efficient Training of Large Language Models. In *Proceedings of the Symposium on Operating Systems Principles (SOSP)*.
- [21] Lei Guan, Dong-Sheng Li, Jiye Liang, Wen-Jian Wang, Ke-shi Ge, and Xicheng Lu. 2024. Advances of Pipeline Model Parallelism for Deep Learning Training: An Overview. *J. Comput. Sci. Technol.* 39 (2024).

- [22] Cong Guo, Rui Zhang, Jiale Xu, Jingwen Leng, Zihan Liu, Ziyu Huang, Minyi Guo, Hao Wu, Shouren Zhao, Junping Zhao, and Ke Zhang. 2024. GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [23] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *arxiv preprint, 2401.14196* (2024).
- [24] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual.
- [25] Zhongzhe Hu, Junmin Xiao, Zheyue Deng, Mingyi Li, Kewei Zhang, Xiaoyang Zhang, Ke Meng, Ninghui Sun, and Guangming Tan. 2022. MegTaiChi: dynamic tensor-based memory management optimization for DNN training. In *International Conference on Supercomputing (ICS)*.
- [26] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [27] Alexander Isenko, Ruben Mayer, Jeffrey Jede, and Hans-Arno Jacobsen. 2022. Where Is My Training Bottleneck? Hidden Trade-Offs in Deep Learning Preprocessing Pipelines. In *Proceedings of the ACM on Management of Data (SIGMOD)*.
- [28] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. *arxiv preprint, 2309.14509* (2023).
- [29] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7B. *arxiv preprint, 2310.06825* (2023).
- [30] Jeff Johnson, Matthijs Douze, and Herv   J  gou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7 (2021).
- [31] Dhiraj D. Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. 2019. A Study of BFLOAT16 for Deep Learning Training. *arxiv preprint, 1905.12322* (2019).
- [32] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Fran  ois Fleuret. 2020. Transformers are RNNs: fast autoregressive transformers with linear attention. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [33] Mahmoud Abo Khamis, Hung Q. Ngo, XuanLong Nguyen, Dan Olteanu, and Maximilian Schleich. 2020. Learning Models over Relational Data Using Sparse Tensors and Functional Dependencies. *ACM Trans. Database Syst.* 45 (2020).
- [34] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2021. Dynamic Tensor Rematerialization. In *International Conference on Learning Representations (ICLR)*.
- [35] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing Activation Recomputation in Large Transformer Models. *arxiv preprint, 2205.05198* (2022).
- [36] Dimitrios Koutsoukos, Supun Nakandala, Konstantinos Karanasos, Karla Saur, Gustavo Alonso, and Matteo Interlandi. 2021. Tensors: An abstraction for general data processing. *Proceedings of the VLDB Endowment* 14 (2021).
- [37] Dacheng Li, Rulin Shao, Anze Xie, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. LightSeq: Sequence Level Parallelism for Distributed Training of Long Context Transformers. *arxiv preprint, 2310.03294* (2023).
- [38] Mu Li, David G. Anderson, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. 2014. Scaling Distributed Machine Learning with the Parameter Server. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [39] Yiming Li, Yanyan Shen, Lei Chen, and Mingxuan Yuan. 2023. Orca: Scalable Temporal Graph Neural Network Training with Theoretical Guarantees. In *Proceedings of the ACM on Management of Data (SIGMOD)*.
- [40] Yikuan Li, Ramsey M. Wehbe, Faraz S. Ahmad, Hanyin Wang, and Yuan Luo. 2022. Clinical-Longformer and Clinical-BigBird: Transformers for long clinical sequences. *arxiv preprint, 2201.11838* (2022).
- [41] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. 2024. World Model on Million-Length Video And Language With Blockwise RingAttention. *arxiv preprint, 2402.08268* (2024).
- [42] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring Attention with Blockwise Transformers for Near-Infinite Context. *arxiv preprint, 2310.01889* (2023).

- [43] Xiaoran Liu, Hang Yan, Chenxin An, Xipeng Qiu, and Dahua Lin. 2024. Scaling Laws of RoPE-based Extrapolation. In *International Conference on Learning Representations (ICLR)*.
- [44] Xupeng Miao, Zhihao Jia, and Bin Cui. 2024. Demystifying Data Management for Large Language Models. In *Proceedings of the ACM on Management of Data (SIGMOD)*.
- [45] Xupeng Miao, Yining Shi, Zhi Yang, Bin Cui, and Zhihao Jia. 2023. SDPipe: A Semi-Decentralized Framework for Heterogeneity-aware Pipeline-parallel Training. *Proceedings of the VLDB Endowment* 16 (2023).
- [46] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proceedings of the VLDB Endowment* 16 (2022).
- [47] Xupeng Miao, Hailin Zhang, Yining Shi, Xiaonan Nie, Zhi Yang, Yangyu Tao, and Bin Cui. 2021. HET: Scaling out Huge Embedding Model Training via Cache-enabled Distributed Framework. *Proceedings of the VLDB Endowment* 15 (2021).
- [48] Xupeng Miao, Shenhan Zhu, Fangcheng Fu, Ziyu Guo, Zhi Yang, Yaofeng Tu, Zhihao Jia, and Bin Cui. 2024. X-former Elucidator: Reviving Efficient Attention for Long Context Language Modeling. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*.
- [49] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *International Conference on Learning Representations (ICLR)*.
- [50] Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. 2021. Analyzing and Mitigating Data Stalls in DNN Training. *Proceedings of the VLDB Endowment* 14 (2021).
- [51] Derek Gordon Murray, Jiri Simsa, Ana Klimovic, and Ihor Indyk. 2021. tf.data: A Machine Learning Data Processing Framework. *Proceedings of the VLDB Endowment* 14 (2021).
- [52] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. 2020. A Tensor Compiler for Unified Machine Learning Prediction Serving. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [53] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-Efficient Pipeline-Parallel DNN Training. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [54] Tung Nguyen, Johannes Brandstetter, Ashish Kapoor, Jayesh K. Gupta, and Aditya Grover. 2023. ClimaX: A foundation model for weather and climate. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [55] NVIDIA. 2024. Context Parallelism. https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/context_parallel.html.
- [56] NVIDIA. 2024. Transformer Engine. <https://github.com/NVIDIA/TransformerEngine>.
- [57] Beng Chin Ooi, Shaofeng Cai, Gang Chen, Kian-Lee Tan, Yuncheng Wu, Xiaokui Xiao, Naili Xing, Cong Yue, Lingze Zeng, Meihui Zhang, and Zhanhao Zhao. 2024. NeurDB: An AI-powered Autonomous Data System. *Science China Information Sciences* 67 (2024).
- [58] OpenAI. 2023. GPT-4 Technical Report. (2023).
- [59] Ismail Oukid, Daniel Booss, Adrien Lespinnasse, Wolfgang Lehner, Thomas Willhalm, and Grégoire Gomes. 2017. Memory management techniques for large-scale persistent-main-memory systems. *Proceedings of the VLDB Endowment* 10 (2017).
- [60] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [61] William Peebles and Saining Xie. 2023. Scalable Diffusion Models with Transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [62] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2024. YaRN: Efficient Context Window Extension of Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [63] Quan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [64] Thomas Phan and Wen-Syan Li. 2008. Dynamic Materialization of Query Views for Data Warehouse Workloads. In *IEEE International Conference on Data Engineering (ICDE)*.
- [65] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of the Conference on Machine Learning and Systems (MLSys)*.

- [66] Ofir Press, Noah A. Smith, and Mike Lewis. 2022. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *International Conference on Learning Representations (ICLR)*.
- [67] Zhen Qin, Weixuan Sun, Hui Deng, Dongxu Li, Yunshen Wei, Baohong Lv, Junjie Yan, Lingpeng Kong, and Yiran Zhong. 2022. cosFormer: Rethinking Softmax In Attention. In *International Conference on Learning Representations (ICLR)*.
- [68] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2019. ZeRO: Memory Optimization Towards Training A Trillion Parameter Models. *arxiv preprint, 1910.02054* (2019).
- [69] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Conference on Knowledge Discovery and Data Mining (KDD)*.
- [70] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. *arxiv preprint, 2101.06840* (2021).
- [71] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *International Symposium on Microarchitecture (MICRO)*.
- [72] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *arxiv preprint, 2308.12950* (2023).
- [73] Ludan Ruan and Qin Jin. 2022. Survey: Transformer based video-language pre-training. *AI Open* 3 (2022).
- [74] Ran Rui, Hao Li, and Yi-Cheng Tu. 2020. Efficient join algorithms for large database tables in a multi-GPU environment. *Proceedings of the VLDB Endowment* 14 (2020).
- [75] Maximilian Schleich, Amir Shaikhha, and Dan Suciu. 2023. Optimizing Tensor Programs on Flexible Storage. In *Proceedings of the ACM on Management of Data (SIGMOD)*.
- [76] Taro Sekiyama, Takashi Imamichi, Haruki Imai, and Rudy Raymond. 2018. Profile-guided memory optimization for deep neural networks. *arxiv preprint, 1804.10001* (2018).
- [77] Mohammad Shoyebi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arxiv preprint, 1909.08053* (2019).
- [78] Benoit Steiner, Mostafa Elhoushi, Jacob Kahn, and James Hegarty. 2022. OLLA: Optimizing the Lifetime and Location of Arrays to Reduce the Memory Usage of Neural Networks. *arxiv preprint, 2210.12924* (2022).
- [79] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html> 3 (2023).
- [80] Together.ai. 2023. LLaMA-2-7B-32K. <https://huggingface.co/togethercomputer/LLaMA-2-7B-32K>
- [81] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arxiv preprint, 2307.09288* (2023).
- [82] Taegeon Um, Byungsoo Oh, Byeongchan Seo, Minhyeok Kweun, Goeun Kim, and Woo-Yeon Lee. 2023. FastFlow: Accelerating Deep Learning Model Training with Smart Offloading of Input Data Pipeline. *Proceedings of the VLDB Endowment* 16 (2023).
- [83] Guozheng Wang, Yongmei Lei, Zeyu Zhang, and Cunlu Peng. 2023. A Communication Efficient ADMM-based Distributed Algorithm Using Two-Dimensional Torus Grouping AllReduce. *Data Sci. Eng.* 8 (2023).
- [84] Kaibo Wang, Kai Zhang, Yuan Yuan, Siyuan Ma, Rubao Lee, Xiaoning Ding, and Xiaodong Zhang. 2014. Concurrent analytical query processing with GPUs. *Proceedings of the VLDB Endowment* 7 (2014).
- [85] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-Attention with Linear Complexity. *arxiv preprint, 2006.04768* (2020).

- [86] Yujie Wang, Youhe Jiang, Xupeng Miao, Fangcheng Fu, Shenhan Zhu, Xiaonan Nie, Yaofeng Tu, and Bin Cui. 2024. Improving Automatic Parallel Training via Balanced Memory Workload Optimization. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 36 (2024).
- [87] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. 2021. Nyströmformer: A Nyström-based Algorithm for Approximating Self-Attention. In *The AAAI Conference on Artificial Intelligence (AAAI)*.
- [88] Feng Yu, Jiacheng Zhao, Hui-Min Cui, Xiaobing Feng, and Jingling Xue. 2023. VTensor: Using Virtual Tensors to Build a Layout-Oblivious AI Programming Framework. *J. Comput. Sci. Technol.* 38 (2023).
- [89] Binhang Yuan, Dimitrije Jankov, Jia Zou, Yuxin Tang, Daniel Bourgeois, and Chris Jermaine. 2021. Tensor Relational Algebra for Distributed Machine Learning System Design. *Proceedings of the VLDB Endowment* 14 (2021).
- [90] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the USENIX Conference on Networked Systems Design and Implementation (NSDI)*.
- [91] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: a unified engine for big data processing. *Commun. ACM* 59 (2016).
- [92] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [93] Hailin Zhang, Zirui Liu, Boxuan Chen, Yikai Zhao, Tong Zhao, Tong Yang, and Bin Cui. 2024. CAFE: Towards Compact, Adaptive, and Fast Embedding for Large-scale Recommendation Models. *Proceedings of the ACM on Management of Data (SIGMOD)*.
- [94] Huangzhao Zhang, Kechi Zhang, Zhuo Li, Jia Li, Jia Li, Yongmin Li, Yunfei Zhao, Yuqi Zhu, Fang Liu, Ge Li, and Zhi Jin. 2024. Deep learning for code generation: a survey. *Science China Information Sciences* 67 (2024).
- [95] Hailin Zhang, Penghao Zhao, Xupeng Miao, Yingxia Shao, Zirui Liu, Tong Yang, and Bin Cui. 2023. Experimental Analysis of Large-scale Learnable Vector Storage Compression. *Proceedings of the VLDB Endowment* 17 (2023).
- [96] Jianhao Zhang, Shihan Ma, Peihong Liu, and Jinhui Yuan. 2023. Coop: Memory is not a Commodity. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [97] Zhen Zhang, Shuai Zheng, Yida Wang, Justin Chiu, George Karypis, Trishul Chilimbi, Mu Li, and Xin Jin. 2022. MiCS: Near-linear Scaling for Training Gigantic Model on Public Cloud. *Proceedings of the VLDB Endowment* 16 (2022).
- [98] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2024. Retrieval-Augmented Generation for AI-Generated Content: A Survey. *arxiv preprint, 2402.19473* (2024).
- [99] Yue Zhao, George H. Chen, and Zhihao Jia. 2022. TOD: GPU-accelerated Outlier Detection via Tensor Operations. *Proceedings of the VLDB Endowment* 16 (2022).
- [100] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* 16 (2023).
- [101] Zangwei Zheng, Xiangyu Peng, Tianji Yang, Chenhui Shen, Shenggui Li, Hongxin Liu, Yukun Zhou, Tianyi Li, and Yang You. 2024. *Open-Sora: Democratizing Efficient Video Production for All*. <https://github.com/hpcaitech/Open-Sora>
- [102] Xuanhe Zhou, Zhaoyan Sun, and Guoliang Li. 2024. DB-GPT: Large Language Model Meets Database. *Data Sci. Eng.* 9 (2024).
- [103] Wenhao Zhu, Hongyi Liu, Qingxiu Dong, Jingjing Xu, Shujian Huang, Lingpeng Kong, Jiajun Chen, and Lei Li. 2024. Multilingual Machine Translation with Large Language Models: Empirical Results and Analysis. In *Findings of the Association for Computational Linguistics (NAACL)*.
- [104] Martin Zinkevich, Markus Weimer, Alexander J. Smola, and Lihong Li. 2010. Parallelized Stochastic Gradient Descent. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Received July 2024; revised September 2024; accepted November 2024